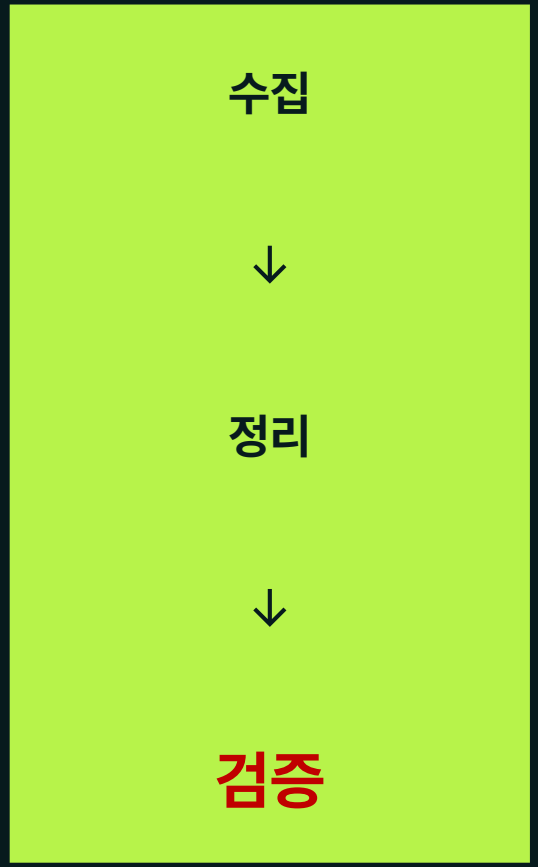
공개 API
데이터 파이프라인

내려받기 버튼 대신, 다시 돌릴 수 있는 수집 코드를 만듭니다



Claude · Claude Code 금융 실무 AI 과정 · 12주차

오늘은 네 기관의 API로 금리차 표와 공시 모니터를 만듭니다

산출물 — 한미 정책금리 차 표·차트, 관심 기업 최근 공시 표, 키가 빠진 git 저장소

01	**API의 기본 지식**	**요청과 응답** 주소의 네 부분 · JSON · 용어 · 커넥터와 차이
02	**네 가지 공개 API**	**DART · 공공데이터 · ECOS · FRED** 기관별 자료 · 키 발급 · 함정 · 실시간 자료 사례
03	**파이프라인 설계**	**수집 · 보관 · 정리 · 산출** 설계 원칙 · 키 유출 사례 · 흔한 오류
04	**실습**	**금리차 · 공시 모니터** ECOS+FRED 금리차 · DART 공시 표 · 확인표

4부 시작 — 9주차 개발환경 위에 '바깥 데이터'를 붙입니다

01

API의 기본 지식

사람이 보는 화면 대신, 프로그램이 읽는 창구로 자료를 받는다

화면에서 내려받던 자료를 API는 코드 한 줄로 받게 합니다

기본 지식 — API(Application Programming Interface): 프로그램끼리 자료를 주고받는 정해진 창구

화면 내려받기

매번 사람이 클릭

- 사이트 접속 · 검색
- 엑셀로 내려받기
- 복사해 붙여 합치기

기간을 바꾸면 처음부터

API 호출

조건만 바꿔 재실행

- 주소에 조건을 담음
- JSON으로 응답
- 바로 표로 정리

내일도 같은 코드로 갱신

※ 같은 자료라도 API는 '언제, 어떤 조건으로 받았는지'가 코드에 남는다. 이것이 감사 가능성의 출발점이다.

API 요청 주소는 기관·자료·조건·신원의 네 부분으로 읽습니다

FRED의 미 국채 10년물(DGS10) 요청을 예로 — 다른 기관도 순서만 다를 뿐 구조는 같다

요청 (Request) — 주소 한 줄에 네 가지가 들어 있다

```
https://api.stlouisfed.org/fred/series/observations?series_id=DGS10&api_key=•••&file_type=json
```

① 기본 주소	② 엔드포인트	③ 파라미터	④ 인증키
어느 기관 서버인가	어떤 자료를 달라는가	조건 (종목 · 기간 · 형식)	누가 부르는가 · 비밀

응답 (Response) — 사람이 아니라 프로그램이 읽는 형식

```
{"observations": [
  {"date": "2026-09-30", "value": "•.••"}, ...]}
```

JSON → pandas 표
날짜 · 값 두 열로 정리

※ 인증키가 주소에 그대로 들어가므로, 주소를 복사해 메신저·메일로 보내면 키도 함께 나간다.

여섯 단어로 API 문서를 읽을 수 있습니다

기관마다 개발 가이드의 표현은 달라도 같은 개념을 가리킨다

용어	뜻	예
엔드포인트 (endpoint)	**자료 종류별 주소**	**list.json · StatisticSearch**
파라미터 (parameter)	**조회 조건**	**기간 · 종목 · 주기**
인증키 (API key)	**호출자 식별 · 한도 관리**	**crtfc_key · api_key**
JSON	**키-값 구조의 응답 형식**	**{"date": ..., "value": ...}**
호출 한도 (rate limit)	**일·분당 최대 요청 수**	**DART 일 2만 건**
상태 코드	**성공·실패 신호**	**000 정상 · 013 자료 없음**

※ 오류는 대부분 화면이 아니라 상태 코드와 메시지로 온다. Claude Code에게 '응답 메시지를 그대로 보여 줘'라고 한다.

8주차 커넥터는 내 업무 앱을, 12주차 API는 기관의 공개 자료를 잇습니다

둘 다 '사람이 나르던 자료를 Claude가 직접 받는다'는 점은 같고, 쓰는 자리와 방식이 다르다

커넥터 (8주차)

대화창 · 내 계정

- Gmail · Drive · Calendar
- 설정에서 연결
- 요청할 때마다 읽음

일회성 조회에 적합

API 스크립트 (12주차)

폴더 · 코드 · 키

- DART · ECOS · FRED
- Python 파일로 저장
- 같은 수집을 반복

정기 갱신 · 기록에 적합

※ 공개 API를 감싼 MCP 서버(예: DART MCP)도 있다. 회사 PC에 설치하기 전 누가 만들었는지 확인하는 원칙은 같다.

02

네 가지 공개 API

공시 · 공공 행정 · 국내 거시 · 해외 거시 — 금융 실무 자료의 네 축

네 기관의 API가 공시·공공·국내 거시·해외 거시 자료를 나눠 맡습니다

모두 무료 · 인증키 필요 (2026년 10월 기준, 한도는 기관 정책에 따라 바뀔 수 있다)

API	제공 기관	대표 자료	키 발급
OpenDART	**금융감독원**	**공시 목록 · 재무제표**	**개인 즉시 · 일 2만 건**
공공데이터포털	행정안전부	주식시세 · 공공기관 자료	API별 활용신청
ECOS	한국은행	금리 · 환율 · 통화량	가입 후 발급
FRED	St. Louis Fed	미 금리 · 물가 · 고용	무료 계정

※ 키는 수업 전에 발급받아 둔다. 기관에 따라 발급 직후 몇 시간은 호출이 막힐 수 있다.

DART는 6자리 종목코드가 아니라 8자리 고유번호로 조회합니다

OpenDART — corpCode.xml(전체 회사 고유번호 zip)을 먼저 받아 종목코드와 짝짓는다

corpCode.xml

고유번호 표

- zip 파일로 응답
- 하루 한 번 캐시

list.json

공시 목록

- 기간 · 회사 조건
- 보고서명 · 접수번호

fnlttSinglAcnt

주요 재무

- 사업연도 · 보고서
- 11011 = 사업보고서

보고서 코드: 11011 사업 · 11012 반기 · 11013 1분기 · 11014 3분기

ECOS 주소는 통계표·주기·기간·항목 코드를 순서대로 이어 붙입니다

…/{통계표}/{주기}/{시작일}/{종료일}/{항목} 순서 — 코드는 ECOS 통계검색 화면에서 찾는다

자료	통계표	주기	항목
한국은행 기준금리	**722Y001**	**D (일)**	**0101000**
원/달러 환율 (매매기준율)	**731Y001**	**D (일)**	**0000001**
주기 표기	**—**	**D · M · Q · A**	**일 · 월 · 분기 · 연**

※ 날짜 형식이 주기마다 다르다(일 20260930, 월 202609). 오류는 RESULT 항목에 코드와 메시지로 온다.

FRED는 계열 ID 하나로 미국 거시 자료를 받습니다

series/observations 엔드포인트 — series_id · api_key · file_type · observation_start

DFEDTARU

연방기금 목표

상단 · 일별

DGS10

국채 10년

시장 금리 · 일별

CPIAUCSL

소비자물가

지수 · 월별

주의

결측 = "."

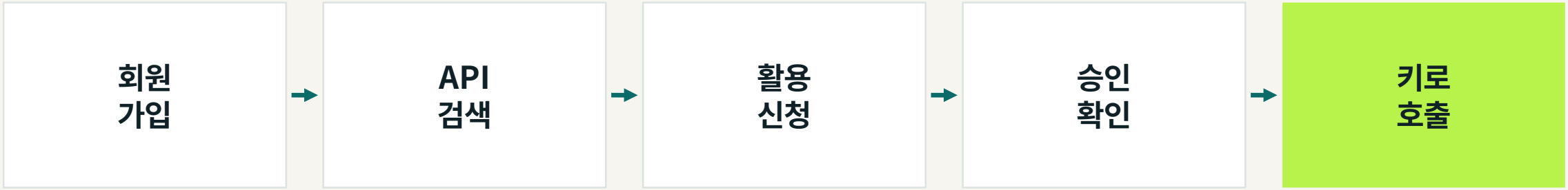
숫자 변환 필수

휴장일 값은 빈칸이 아니라 마침표(.)로 옵니다.

그대로 평균을 내면 오류가 나거나, 문자로 처리되어 조용히 빠집니다.

공공데이터포털은 API마다 활용신청을 하고 키 두 종류 중 맞는 것을 씁니다

data.go.kr — 예: 금융위원회 주식시세정보, 공공기관 경영정보 등. 자동승인은 즉시, 심사형은 영업일 기준



가장 흔한 오류

SERVICE_KEY_IS_NOT_REGISTERED — 인코딩·디코딩 키를 바꿔 본다

※ requests는 주소를 자동으로 인코딩하므로 보통 '디코딩 키'를 넣는다. 이미 인코딩된 키를 넣으면 이중 인코딩이 된다.

지금 받은 거시 자료는 당시 사람들이 본 숫자와 다를 수 있습니다

사례 — Orphanides(2001, AER): 사후 수정된 자료로 평가한 1970년대 통화정책 판단이 실시간 자료로는 달라졌다

무엇이 문제인가

자료는 수정된다

- GDP · 고용은 여러 번 개정
- 오늘 받은 과거 값 ≠ 당시 값
- 백테스트가 실제보다 좋아짐

미래 정보가 섞인다 (look-ahead)

파이프라인 대처

받은 날짜를 남긴다

- 원본을 수집일별 보관
- 수정 이력은 ALFRED
- 공시는 정정 여부 확인

언제 알 수 있었나를 기록

※ DART도 같다. 정정공시가 나오면 같은 사업연도 재무 숫자가 바뀐다. 기준일 없는 숫자는 보고서에 쓰지 않는다.

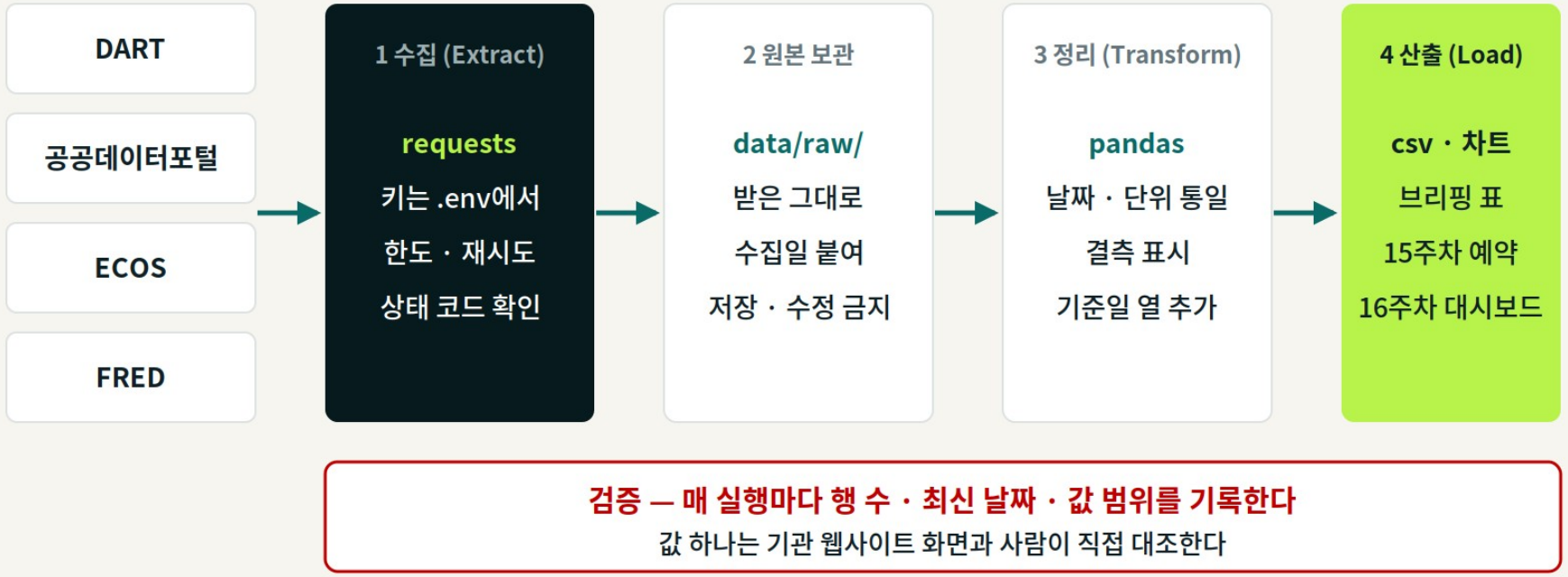
03

파이프라인 설계

한 번 받고 끝나는 코드가 아니라, 매주 다시 돌려도 같은 품질이 나오는 코드

파이프라인은 수집·보관·정리·산출 네 단계와 매번의 검증으로 이뤄집니다

기본 지식 — ETL(Extract · Transform · Load): 데이터 공학의 표준 흐름. 원본 보관을 따로 둔다



※ 15주차에는 이 파이프라인을 예약 작업으로 매일 돌리고, 16주차에는 산출물을 대시보드로 띄운다.

좋은 수집 코드는 네 가지 원칙을 지킵니다

이 원칙을 CLAUDE.md에 적어 두면 Claude Code가 새 스크립트를 쓸 때마다 따른다

1

원본 보관

받은 그대로

2

멱등성

다시 돌려도 같음

3

기준일 열

날짜 · 단위 표기

4

실행 기록

행 수 · 최신일

멱등성(idempotency) — 같은 날 두 번 돌려도 자료가 두 배로 쌓이지 않아야 합니다.

인증키가 코드와 함께 공개되면 몇 년간 모르고 지날 수 있습니다

사례 — 2022년 10월 Toyota: GitHub에 약 5년간 공개된 소스코드 안에 서버 접속 키가 있었다

무슨 일이었나

외주 개발자의 실수

- 소스 일부를 공개 저장소에
- 코드 안에 접속 키
- 고객 약 30만 명 메일 노출 우려

키 교체로 수습 · 5년간 미인지

우리 폴더의 규칙

키는 .env에만

- .env는 .gitignore에
- 코드에는 변수 이름만
- .env.example로 형식 공유

"키가 커밋에 없는지 확인해 줘"

※ 공개 API 키는 무료라도 한도가 남에게 소진되거나 계정이 정지될 수 있다. 유출을 알면 즉시 재발급한다.

오류의 대부분은 키·한도·형식 세 가지에서 나옵니다

오류 메시지를 Claude Code에 그대로 보여 주면 원인을 함께 좁힌다

증상	흔한 원인	대처
인증 오류 · 미등록 키	키 오타 · 이중 인코딩	디코딩 키 · .env 확인
DART 020	일 호출 한도 초과	캐시 · 반복 실행 줄이기
DART 013 · 빈 응답	기간에 자료 없음	정상 — 0건으로 기록
숫자가 문자로 남음	"." · 쉼표 · 공백	to_numeric 변환
값이 100배 차이	단위 (원/억원 · %/bp)	단위 열 추가 · 대조

※ 'Excel로 받은 값과 API 값이 다르다'면 대부분 기준일·단위·수정 시점 차이다. 코드보다 정의부터 맞춘다.

실습 지표는 두 계열의 차와 전년 동기 대비 변화율입니다

두 계열의 날짜를 먼저 맞춘다 — 정책금리는 결정일 사이를 직전 값으로 채운다(forward fill)

한미 정책금리 차 (%p)

$$\Delta_t = i_t^{\mathrm{BOK}} - i_t^{\mathrm{Fed}}$$

전년 동기 대비 (월별 자료)

$$g_t = \frac{X_t}{X_{t-12}} - 1$$

Fed는 목표 범위로 발표하므로 상단(DFEDTARU)을 쓴다고 표에 적습니다.

정의를 적지 않은 지표는 같은 이름이라도 다른 숫자가 됩니다.

04

실습: 금리차와 공시 모니터

실습 폴더에서 키만 넣고, 나머지는 Claude Code에게 말로 시킨다

금리차 실습은 정의·원본 보관·대조를 함께 요청합니다

실습 1 — Week12_Lab_Files 폴더, .env에 ECOS·FRED 키 입력 후

Claude Code에게 이렇게 말합니다

"ECOS 기준금리(722Y001)와 FRED 연방기금 목표 상단(DFEDTARU)으로 2020년 이후 한미 정책금리 차 표와 차트를 만들어 줘.
응답 원본은 data/raw에 날짜를 붙여 보관하고,
최신 값은 내가 웹에서 대조할 수 있게 날짜와 함께 따로 보여 줘."

Claude Code가 실행하는 명령

```
pip install requests pandas
(rate_gap.py 등 작성)
python rate_gap.py 2020-01-01
→ data/out/rate_gap.csv
→ data/out/rate_gap.png
→ data/out/run_log.txt
```

공시 모니터는 고유번호 변환과 0건 처리를 먼저 확인합니다

실습 2 — watchlist.csv에 관심 기업 5곳(회사명, 종목코드)

Claude Code에게 이렇게 말합니다

"watchlist.csv 기업들의 최근 7일 DART 공시를
접수일·회사·보고서명·링크 표로 만들어 줘.
종목코드는 corpCode.xml로 고유번호로 바꾸고,
못 찾은 종목과 공시 0건인 회사도
결과에 따로 알려 줘."

Claude Code가 실행하는 명령

```
(dart_monitor.py 작성)
python dart_monitor.py 7
→ corpCode.xml 받기 • 캐시
→ list.json × 5회
→ data/out/dart_YYYYMMDD.csv
```

호출 횟수를 줄이는 캐시가 한도 초과(020)를 막습니다.

실습이 끝나면 네 가지를 확인하고 기록합니다

확인표 — 숫자보다 '다시 돌릴 수 있고, 키가 새지 않는가'를 본다

항목	확인할 것	방법
키 보호	**커밋 이력에 키 문자열 없음**	**git log -p 검색 요청**
원본 보관	**data/raw에 수집일별 파일**	**폴더 목록 확인**
숫자 대조	**최신 금리차 = 웹 화면 값**	**ECOS · FRED 화면**
재실행	**두 번 돌려도 행 수 같음**	**run_log.txt 비교**

※ 실습 스크립트는 기관 문서의 응답 형식으로 점검했다. 실제 값은 실행 날짜와 기관 개정에 따라 달라진다.

오늘 만든 수집 코드가 13–15주차의 재료가 됩니다

4부 데이터 파이프라인 — 수집(12) → 양식(13) → 웹 수집·LLM(14) → 에이전트(15)

12주
공개 API
수집 코드

13주
양식 엑셀
월 집계 · 매핑

14주
웹 수집
크롤링 · LLM

15주
에이전트
예약 · 뉴스레터

지금은 사람이 rate_gap.py를 직접 실행합니다.

15주차에는 매일 아침 스스로 돌고, 결과를 브리핑으로 보냅니다.

다음 주까지 부서가 매주 받는 지표 하나를 API 수집 코드로 바꿉니다

과제는 코드보다 '정의 · 원본 · 대조 기록'이 함께 있는지를 본다

과제 1

지표 하나 자동화

정의 · 단위 명시

과제 2

대조 기록

웹 화면 캡처 1장

13주차 예고

양식 엑셀 채우기

ECOS → 보고 양식

13주차에는 오늘 받은 ECOS·FRED 자료를 부서 보고 양식에 채웁니다.

CODE THE DOWNLOAD.

KEEP THE RAW, DATE EVERYTHING.

KEYS NEVER TOUCH GIT.

받은 날짜가 없는 숫자는
보고서에 쓰지 않습니다.

12주차 · 공개 API 파이프라인

다음 주 — 13주차 보고 양식 엑셀 채우기: 월 집계 · 매핑 · 서식 보존